

1 Phân tích Bài toán Pipe Puzzle

Bài toán được mô phỏng trên một lưới chỉnh hợp kích thước $M \times N$. Mỗi ô trong lưới chứa một đoạn ống nước có khả năng xoay các góc $0^\circ, 90^\circ, 180^\circ, 270^\circ$.

Các ràng buộc cơ bản của bài toán bao gồm:

- **Tính liên thông:** Nguồn nước (Source) phải kết nối được với tất cả các ô trên lưới thông qua hệ thống ống.
- **Tính khép kín:** Không tồn tại đầu ống nào hướng ra ngoài biên hoặc hướng vào cạnh của một ô lân cận mà tại đó không có đầu ống nối tương ứng.

Mục tiêu là tìm ra cấu hình xoay cho tất cả các ô sao cho thỏa mãn đồng thời các ràng buộc trên.

2 Hiện thực Thuật toán DFS

Để giải quyết bài toán, ta tiếp cận bằng thuật toán Tìm kiếm theo chiều sâu (Depth-First Search - DFS) kết hợp quay lui (Backtracking). Thuật toán duyệt qua từng ô theo thứ tự tuyến tính từ 0 đến $(M \times N - 1)$.

2.1 Mã giả cơ bản

Dưới đây là cấu trúc cơ bản của thuật toán khi chưa áp dụng tối ưu hóa:

Algorithm 1: Giải thuật DFS cho Pipe Puzzle

```
1 FUNCTION DFS(grid, cell_index):
2   if cell_index == Total_Cells then
3     return IsGridValid(grid)           // Kiểm tra tính hợp lệ toàn cục
4   (row, col) ← GetCoordinates(cell_index)
5   for rotation ∈ {0, 1, 2, 3} do
6     grid[row][col].current_rotation ← rotation
7     if DFS(grid, cell_index + 1) then
8       return true
9   return false
```

2.2 Đánh giá độ phức tạp

Về mặt lý thuyết, với mỗi ô có 4 trạng thái xoay, không gian tìm kiếm sẽ bùng nổ theo hàm mũ: $O(4^{M \times N})$. Với một lưới kích thước 5×5 , số trạng thái cần xét lên đến $4^{25} \approx 1.12 \times 10^{15}$, vượt quá khả năng xử lý của các hệ thống máy tính thông thường trong thời gian cho phép.

3 Tối ưu hóa bằng Kỹ thuật Pruning (Cắt tỉa)

Để giảm thiểu không gian tìm kiếm, ta thực hiện kiểm tra các ràng buộc cục bộ ngay trong quá trình đệ quy thay vì đợi đến khi hoàn thành toàn bộ lưới.

Ràng buộc biên (Boundary Constraints): Các ô ở góc hoặc cạnh chỉ có một số ít trạng thái xoay hợp lệ (ví dụ: ống chữ L ở góc chỉ có 1 hướng xoay không đâm vào tường). Ta chỉ xét các *rotation* thỏa mãn điều kiện này.

Ràng buộc kết nối lân cận (Neighbor Matching): Khi xét ô tại vị trí t , các ô lân cận có chỉ số nhỏ hơn t (ô phía trên và bên trái) đã được cố định trạng thái.

- Nếu ô phía trên có đầu nối hướng xuống, ô hiện tại **bắt buộc** phải có đầu nối hướng lên.
- Nếu ô phía trên không có đầu nối hướng xuống, ô hiện tại **không được phép** có đầu nối hướng lên.

Việc áp dụng các điều kiện cắt tỉa này giúp giảm số lượng trạng thái khả thi tại mỗi bước từ 4 xuống còn 1 hoặc thậm chí bằng 0 (dẫn đến quay lui ngay lập tức), giúp thuật toán hội tụ về lời giải nhanh hơn đáng kể.

4 Kết quả thực nghiệm và Đánh giá

Dưới đây là bảng số liệu thu được sau khi tiến hành thực nghiệm thuật toán DFS (có áp dụng kỹ thuật Pruning) trên các lưới kích thước khác nhau với cùng cấu hình FPS = 10.

Bảng 1: Kết quả đo lường hiệu năng của thuật toán DFS với Pruning

Sample	4x4		6x6		8x8	
	Time (s)	Mem (B)	Time (s)	Mem (B)	Time (s)	Mem (B)
1	6.8382	6408	4.1827	12216	11.5036	42158
2	1.0089	7258	11.2086	11900	1032.62	26150
3	1.8336	5804	15.9486	12918	13.5933	46790
4	0.8527	5788	3.2414	12958	398.9448	26110
5	1.3088	6024	91.7239	12878	5.2527	48166
Average	2.36844	6256.4	25.26104	12574	292.3829	37874.8
Variance	6.383365	376064.8	1407.666	235162	199591.7	119904211

4.1 Phân tích số liệu

Dựa trên kết quả thực nghiệm tại Bảng 1, ta có các nhận xét sau:

- **Về thời gian thực thi (Time):** Thời gian xử lý trung bình tăng trưởng theo hàm mũ khi kích thước lưới mở rộng. Đặc biệt, tại lưới 8×8 , phương sai (Variance) đạt mức rất cao (199,591.7). Điều này cho thấy hiệu quả của kỹ thuật cắt tỉa phụ thuộc cực lớn vào cấu trúc của từng mẫu thử (Input-dependent). Nếu các nhánh sai bị cắt bỏ sớm, thuật toán hội tụ rất nhanh; ngược lại, hệ thống sẽ rơi vào trạng thái bùng nổ tổ hợp.
- **Về bộ nhớ tiêu thụ (Memory):** Trái ngược với thời gian, bộ nhớ tăng trưởng khá ổn định và tuyến tính theo số lượng ô trên lưới ($M \times N$). Điều này minh chứng cho ưu điểm của giải thuật DFS trong việc quản lý bộ nhớ, khi chỉ cần lưu trữ trạng thái trên nhánh đang duyệt thay vì toàn bộ không gian trạng thái.

- **Đánh giá kỹ thuật Pruning:** Kỹ thuật cắt tỉa đã giúp bài toán khả thi hơn trên các lưới nhỏ, nhưng đối với các lưới lớn (8×8 trở lên), cần có thêm các ràng buộc thông minh hơn hoặc áp dụng các thuật toán heuristic để ổn định thời gian thực thi.